



Universidad
de Cartagena
Fundada en 1827

CTEV Centro tecnológico
para la formación
virtual y a distancia
Universidad de Cartagena

Computación, procesadores y memoria

Descripción: Este módulo introduce a los estudiantes a los principios fundamentales de la ingeniería de software, la industria del software y los sistemas de computación. Los estudiantes explorarán la evolución de la arquitectura de la computación, así como los periféricos y sistemas de entrada de información utilizados en la ingeniería de software. Además, los estudiantes aplicarán estos conocimientos en un proyecto práctico.

Objetivos de aprendizaje:

- Comprender los principios fundamentales de la ingeniería de software
- Conocer la industria del software y las tendencias actuales
- Entender la arquitectura de la computación y sus tendencias evolutivas
- Comprender los periféricos y sistemas de entrada de información utilizados en la ingeniería de software

Contenido del módulo:

1. Principios de la ingeniería de software:
 - 1.1 Definición de ingeniería de software
 - 1.2 Principios de la ingeniería de software
 - 1.3 Enfoques de desarrollo de software
2. La industria del software:
 - 2.1 Historia de la industria del software
 - 2.2 Tendencias actuales en la industria del software
 - 2.3 Modelos de negocio en la industria del software
3. Computación, procesadores y memoria:
 - 3.1 Arquitectura de la computación
 - 3.2 Procesadores y memoria
 - 3.3 Tendencias evolutivas en la arquitectura de la computación
4. Periféricos y sistemas de entrada de información:
 - 4.1 Periféricos utilizados en la ingeniería de software
 - 4.2 Sistemas de entrada de información utilizados en la ingeniería de software
 - 4.3 Tendencias en periféricos y sistemas de entrada de información
5. La evolución de la arquitectura de la computación:
 - 5.1 Evolución de la arquitectura de la computación
 - 5.2 Tipos de arquitectura de la computación
 - 5.3 Tendencias actuales en la arquitectura de la computación

1. Principios de la Ingeniería de Software

El desarrollo de software es una disciplina que requiere un enfoque estructurado y metodológico para garantizar la calidad y eficiencia en la creación de sistemas y aplicaciones. En este módulo, exploraremos los principios fundamentales de la ingeniería de software, así como los enfoques comunes utilizados en el desarrollo de software.

Definición de Ingeniería de Software:

La ingeniería de software es una disciplina que se enfoca en la aplicación de principios y métodos de la ingeniería para el desarrollo de software de calidad.

Implica la aplicación sistemática de enfoques, técnicas y herramientas para el diseño, implementación, prueba, mantenimiento y gestión de software.

Principios de la Ingeniería de Software:

Principios clave de la ingeniería de software:

Abstracción y modularidad: Divide el software en componentes más pequeños y manejables para facilitar el desarrollo, mantenimiento y reutilización.

Separación de preocupaciones: Divide las funcionalidades y responsabilidades del software en módulos independientes para mejorar la claridad y facilitar los cambios.

Ocultamiento de información: Protege los detalles internos de los componentes del software para reducir la complejidad y mejorar la seguridad y la interoperabilidad.

Principios adicionales:

Cohesión: Los componentes de software deben estar relacionados de manera lógica y tener una funcionalidad coherente dentro del sistema.

Acoplamiento: Los componentes de software deben estar interconectados de manera eficiente y mínimamente dependiente para promover la flexibilidad y la modularidad.

Enfoques de Desarrollo de Software:

Modelos tradicionales:

Modelo en cascada: Un enfoque secuencial en el que el desarrollo de software se divide en etapas distintas, como análisis, diseño, implementación, pruebas y mantenimiento.

Modelo en espiral: Un enfoque iterativo que combina elementos del enfoque en cascada con la evaluación y mitigación de riesgos.

Metodologías ágiles:

Scrum: Un marco de trabajo ágil que se centra en la entrega iterativa y colaborativa, con equipos multifuncionales y roles definidos.

Kanban: Un enfoque visual que se basa en el flujo continuo de trabajo, limitando el trabajo en progreso y maximizando la eficiencia.

Extreme Programming (XP): Una metodología que se centra en la comunicación constante, la retroalimentación rápida y el desarrollo orientado a pruebas.

2. La Industria del Software

2.1 Historia de la industria del software:

Los inicios de la industria del software se remontan a la década de 1940, cuando los primeros computadores electrónicos empezaron a ser utilizados con fines científicos y militares.

En la década de 1950, se desarrollaron los primeros lenguajes de programación de alto nivel, como el Fortran y el COBOL, lo que facilitó la escritura de software.

En la década de 1960, surgieron las primeras compañías especializadas en el desarrollo de software, como IBM y Microsoft.

La década de 1970 marcó la llegada de los microprocesadores y los primeros sistemas operativos, lo que permitió el crecimiento exponencial de la industria del software.

En la década de 1980, se popularizaron las computadoras personales y el software se convirtió en un elemento esencial para su funcionamiento.

En la década de 1990, con la expansión de Internet, se produjo un gran avance en el desarrollo de software orientado a la web y surgieron empresas como Google y Amazon.

En los últimos años, la industria del software ha experimentado un rápido crecimiento y evolución, impulsada por tecnologías como la inteligencia artificial, la computación en la nube y el Internet de las cosas.

2.2 Tendencias actuales en la industria del software:

Inteligencia artificial y aprendizaje automático: El uso de algoritmos y modelos de aprendizaje automático ha revolucionado la industria del software, permitiendo aplicaciones como el reconocimiento de voz, la traducción automática y los chatbots.

Computación en la nube: La computación en la nube ha facilitado el acceso a recursos de almacenamiento y procesamiento de datos a través de Internet, lo que ha impulsado el desarrollo de aplicaciones y servicios escalables.

Internet de las cosas (IoT): La interconexión de dispositivos físicos a través de Internet ha dado lugar a la creación de software para controlar y gestionar estos dispositivos, desde electrodomésticos inteligentes hasta ciudades inteligentes.

Desarrollo móvil: Con el crecimiento de los teléfonos inteligentes y las aplicaciones móviles, el desarrollo de software se ha enfocado en la creación de aplicaciones para dispositivos móviles, tanto para plataformas iOS como Android.

DevOps: La integración de las operaciones de desarrollo y despliegue de software ha dado lugar a la metodología DevOps, que busca una entrega continua y eficiente de software, mejorando la colaboración entre los equipos de desarrollo y operaciones.

2.3 Modelos de negocio en la industria del software:

Licencia de software: El modelo tradicional de venta de software mediante licencias, donde los usuarios adquieren una licencia de uso del software y pueden instalarlo en sus dispositivos.

Software como servicio (SaaS): Los usuarios acceden al software a través de Internet, generalmente mediante una suscripción, y los proveedores se encargan del mantenimiento, actualizaciones y gestión de la infraestructura.

Modelo freemium: Se ofrece una versión gratuita con funcionalidades básicas y se cobran tarifas por características adicionales o una versión premium del software.

Modelo de código abierto: El software se distribuye con una licencia que permite el acceso al código fuente, lo que fomenta la colaboración y permite a los usuarios modificar y mejorar el software.

Modelo de publicidad: El software se ofrece de forma gratuita, pero incluye anuncios publicitarios que generan ingresos para el desarrollador.

Modelo de desarrollo personalizado: Empresas de software que ofrecen servicios de desarrollo personalizado para satisfacer las necesidades específicas de los clientes, creando soluciones a medida.

3. Computación, Procesadores y Memoria

3.1 Arquitectura de la computación:

La arquitectura de la computación se refiere a la estructura y organización de los componentes principales de un sistema computacional.

La arquitectura de Von Neumann es el modelo clásico de arquitectura de computadoras, que consta de una unidad central de procesamiento (CPU), memoria principal, dispositivos de entrada y salida, y una interconexión entre ellos.

La CPU es responsable de ejecutar instrucciones y consta de una unidad de control y una unidad aritmético-lógica.

La memoria principal almacena datos y programas que se utilizan durante la ejecución del software.

Los dispositivos de entrada y salida permiten la comunicación entre la computadora y el mundo exterior.

3.2 Procesadores y memoria:

Los procesadores, también conocidos como unidades centrales de procesamiento (CPU), son el corazón de una computadora y realizan las operaciones y cálculos necesarios para ejecutar software.

Los procesadores modernos están compuestos por múltiples núcleos, lo que permite la ejecución simultánea de múltiples hilos de instrucciones.

La memoria es el componente donde se almacenan los datos y las instrucciones que el procesador necesita para realizar su trabajo.

La memoria se divide en diferentes niveles, como la memoria caché (caché L1, L2, L3), que se encuentra cerca del procesador y almacena datos e instrucciones que se acceden con mayor frecuencia, y la memoria principal, que es más grande pero más lenta.

La memoria secundaria, como los discos duros y las unidades de estado sólido (SSD), se utiliza para el almacenamiento a largo plazo de datos y programas.

3.3 Tendencias evolutivas en la arquitectura de la computación:

Computación paralela: La tendencia actual es aprovechar la capacidad de los procesadores multinúcleo y distribuir tareas para realizar cálculos de manera simultánea y más eficiente.

Arquitecturas heterogéneas: Se combinan diferentes tipos de procesadores y aceleradores, como unidades de procesamiento gráfico (GPU) y unidades de procesamiento tensorial (TPU), para optimizar el rendimiento en tareas específicas.

Memoria de acceso no volátil (NVM): Las tecnologías de memoria emergentes, como la memoria de cambio de fase (PCM) y la memoria de resistencia variable (RRAM), ofrecen una mayor capacidad y velocidad en comparación con las tecnologías tradicionales, como la memoria RAM.

Computación en la nube: La arquitectura de la computación en la nube permite acceder a recursos informáticos de manera remota a través de Internet, lo que proporciona escalabilidad y flexibilidad en la entrega de servicios y aplicaciones.

4. Periféricos y Sistemas de Entrada de Información

4.1 Periféricos utilizados en la ingeniería de software:

Teclado: El teclado es uno de los periféricos más comunes utilizado en la ingeniería de software para introducir texto y comandos en un sistema informático.

Ratón: El ratón es ampliamente utilizado para interactuar con interfaces gráficas y realizar acciones como hacer clic, arrastrar y soltar.

Monitor: El monitor o pantalla muestra la información visual generada por el software, permitiendo a los ingenieros de software visualizar y analizar datos, así como desarrollar y depurar aplicaciones.

Impresora: Las impresoras se utilizan para generar copias físicas de documentos y código fuente, facilitando la revisión y el análisis en papel.

Escáner: Los escáneres se utilizan para digitalizar documentos en papel, como diagramas y especificaciones, y convertirlos en formato digital para su uso en el desarrollo de software.

Dispositivos de almacenamiento externo: Estos periféricos, como unidades flash USB y discos duros externos, se utilizan para realizar copias de seguridad, transferir y almacenar datos importantes relacionados con el desarrollo de software.

4.2 Sistemas de entrada de información utilizados en la ingeniería de software:

Micrófono: Los micrófonos se utilizan para ingresar información de voz y realizar tareas como dictado de texto o comandos de voz en entornos de desarrollo y sistemas de reconocimiento de voz.

Lápiz óptico y tableta gráfica: Estos dispositivos permiten a los ingenieros de software dibujar y diseñar interfaces gráficas, diagramas y esquemas de manera más precisa y natural.

Dispositivos de captura de movimiento: Estos sistemas, como los sensores de movimiento o las cámaras de captura de movimiento, se utilizan para ingresar datos de movimiento en aplicaciones de realidad virtual, simulación o animación.

4.3 Tendencias en periféricos y sistemas de entrada de información:

Dispositivos de realidad virtual (VR) y realidad aumentada (AR): Estos periféricos y sistemas de entrada, como gafas de realidad virtual y controladores hápticos, permiten una interacción más inmersiva con aplicaciones y entornos virtuales.

Dispositivos de reconocimiento biométrico: Los sistemas de entrada que utilizan huellas dactilares, reconocimiento facial o escaneo de iris están ganando popularidad para autenticación y seguridad en el acceso a sistemas de desarrollo de software.

Dispositivos de voz y asistentes virtuales: Los sistemas de entrada basados en voz, como los asistentes virtuales, se están volviendo más comunes en la interacción con aplicaciones y sistemas de desarrollo, permitiendo tareas como dictado de código y comandos de voz.

Interfaces táctiles y gestuales: Las pantallas táctiles y los sensores de gestos ofrecen una forma intuitiva de interactuar con aplicaciones y sistemas, especialmente en dispositivos móviles y tablets.

5. La Evolución de la Arquitectura de la Computación

5.1 Evolución de la arquitectura de la computación:

La arquitectura de la computación ha experimentado una evolución significativa a lo largo de los años, desde los primeros computadores hasta los sistemas modernos. Algunos hitos importantes incluyen:

La arquitectura de Von Neumann: Propuesta en la década de 1940, estableció los principios fundamentales de la arquitectura de computadoras, incluyendo la separación de la unidad de control y la memoria, el almacenamiento de instrucciones y datos en la memoria y la ejecución secuencial de instrucciones.

Transistores y circuitos integrados: La invención de los transistores en la década de 1950 permitió la miniaturización de los componentes electrónicos y la creación de circuitos integrados, lo que llevó a un aumento significativo en la capacidad de procesamiento y la reducción del tamaño de las computadoras.

Microprocesadores: En la década de 1970, se introdujeron los primeros microprocesadores, que integraban la CPU en un solo chip. Esto permitió la creación de computadoras personales más accesibles y potentes.

Arquitectura RISC: En la década de 1980, surgieron los procesadores con Conjunto de Instrucciones Reducidas (RISC), que simplificaron y agilizaron la ejecución de instrucciones, mejorando el rendimiento de los sistemas.

Paralelismo y procesadores multicore: A medida que la velocidad de los procesadores individuales alcanzaba sus límites físicos, se adoptó el paralelismo en la arquitectura de computadoras, con la incorporación de múltiples núcleos en un solo procesador para realizar tareas en paralelo.

Computación en la nube: La arquitectura de la computación en la nube ha revolucionado la forma en que se proporcionan y consumen los recursos informáticos, permitiendo el acceso bajo demanda a servicios y aplicaciones a través de Internet.

5.2 Tipos de arquitectura de la computación:

Arquitectura de un solo bus: En este tipo de arquitectura, todos los componentes comparten un único bus de datos y control para la comunicación.

Arquitectura cliente-servidor: En esta arquitectura, los clientes solicitan servicios o recursos a un servidor centralizado, que los proporciona y coordina.

Arquitectura basada en microservicios: Esta arquitectura se basa en la construcción de aplicaciones como un conjunto de servicios independientes y altamente acoplados, que se comunican entre sí a través de interfaces bien definidas.

Arquitectura distribuida: En esta arquitectura, los componentes de un sistema están distribuidos en múltiples computadoras interconectadas, lo que permite el procesamiento y almacenamiento distribuido de datos y aplicaciones.

Arquitectura orientada a servicios (SOA): En esta arquitectura, los servicios se ofrecen como componentes independientes y reutilizables, que pueden ser combinados para construir aplicaciones complejas.

5.3 Tendencias actuales en la arquitectura de la computación:

Arquitecturas sin servidor (Serverless): En este enfoque, los desarrolladores se centran en escribir funciones o fragmentos de código que se ejecutan de forma independiente, sin la necesidad de administrar la infraestructura subyacente.

Computación cuántica: La arquitectura de computación cuántica aprovecha los principios de la física cuántica para realizar cálculos complejos de manera más eficiente que los computadores clásicos.

Arquitecturas de contenedores: Los contenedores ofrecen un entorno aislado y portátil para ejecutar aplicaciones y servicios, lo que facilita el despliegue y la escalabilidad de las aplicaciones en diferentes entornos.

Computación en el borde (Edge computing): Esta tendencia busca llevar el procesamiento y almacenamiento de datos más cerca de los dispositivos y usuarios finales, reduciendo la latencia y mejorando la eficiencia en la transmisión de datos.

Arquitecturas autónomas: Se están explorando arquitecturas que permitan a los sistemas informáticos tomar decisiones de forma autónoma, mediante el uso de algoritmos de aprendizaje automático y técnicas de inteligencia artificial.

Referencias:

- Fernández Sanz, L. (2000). El futuro de la ingeniería del software o ¿cuándo será el software un producto de ingeniería? Novática: Revista de la Asociación de Técnicos de Informática, (145), 82. ISSN 0211-2124.
<https://dialnet.unicartagenaproxy.elogim.com/servlet/articulo?codigo=2821868>
- Sommerville, I. (2011). Ingeniería de software (9a ed.). Pearson Educación.
<https://dialnet.unicartagenaproxy.elogim.com/servlet/libro?codigo=323317>
- Pressman, R. (2014). Ingeniería del software: un enfoque práctico. McGraw-Hill.
- Sommerville, I. (2010). Ingeniería del software. Addison-Wesley. Enfoques de desarrollo de software:
- Cockburn, A. (2002). Agile software development. Addison-Wesley.
- Software.org (2019). The software-enabled world. Disponible en:
<https://www.software.org/software-enabled-world/>
- Patterson, D., & Hennessy, J. (2013). Computer organization and design: the hardware/software interface. Morgan Kaufmann.
- Hennessy, J., & Patterson, D. (2018). Computer architecture: a quantitative approach. Morgan Kaufmann.



Universidad
de Cartagena
Fundada en 1827

CTEV Centro tecnológico
para la formación
virtual y a distancia
Universidad de Cartagena